

第9章

关系数据库存储管理

本章讨论关系数据库的数据组织和存储管理。我们将重点介绍基于磁盘的数据库的组织与存储，有关内存数据库的数据组织见新技术篇的第17章。

本章导读

存储管理是数据库管理系统(DBMS)的重要职责之一。它与查询优化以及数据库物理设计都密切相关。

本章介绍关系数据库的数据组织和索引结构。在9.1节数据组织中主要讨论数据库的逻辑与物理组织方式，重点介绍记录的表示、记录如何在块中组织存储以及如何组织关系表的存放；在9.2节索引结构中重点介绍5类索引，包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引。

本章的难点是理解数据库逻辑组织与物理组织之间的对应关系，了解关系表的几种存放方式，深入理解索引的作用，掌握B+树索引和哈希索引的组织方式和查找方式。

在第一篇基础篇中我们已经知道，关系数据库的一个重要特点是存储结构和存取路径对用户隐蔽，用户访问数据库时，只需要用SQL语句表达自己需要哪些信息，比如“查询数据库系统概论课程的选课人数”，而不需要像过程化语言那样具体指定怎么去查找。将用户的查询请求转化成查询执行计划，是数据库查询优化的一个重要工作，而了解关系数据库的数据组织与存储管理，能帮助我们更好地理解关系数据库查询优化的原理。同时，了解数据组织与存储管理，也能帮助我们在应用开发中更好地进行数据库物理设计。

9.1 数据组织

数据库是大量数据的有结构的综合性集合。如何将这样一个庞大的数据集合以最优的形式组织起来存放在外存上，是一个非常重要的问题。所谓“优”应包括两方面：一是存储效率高，节省存储空间；二是存取效率高，访问速度快，代价小。本节主要介绍数据库的逻辑组织方式和物理存储方式。

9.1.1 数据库的逻辑组织方式与物理组织方式

数据库数据存放的基础是文件，对数据库的任何操作最终都要转化为对文件的操作。所以在数据库的物理组织中，基本问题就是如何利用操作系统提供的基本文件组织来设计数据库数据的存放方法，这实际上也就对应了数据存储管理的两种方式。

第一种方式是每一个数据库对象（例如每一个基本表、索引等）都对应一个操作系统的文件，这种数据组织方式本质上是存储管理交由操作系统去完成。PostgreSQL、Kingbase ES 等采用此种存储管理方式。

第二种方式是整个数据库对应一个或若干个文件，由数据库管理系统进行存储管理，这种存储方式也称作段页式存储方式。Oracle、SQL Server 等采用这种存储管理方式。

在段页式存储管理中，数据库管理系统首先向操作系统申请一个大文件，当数据量不断增加，文件空间不够时，数据库管理系统会向操作系统申请追加新的文件。为了方便管理，数据库管理系统通常会对数据库空间进行逻辑划分，以增加灵活性。不同数据库管理系统从逻辑上划分数据库空间的方法并不完全一样，一种常见的划分方式是将数据库组织成“表空间-段-分区-数据块”的形式，如图 9.1 所示。

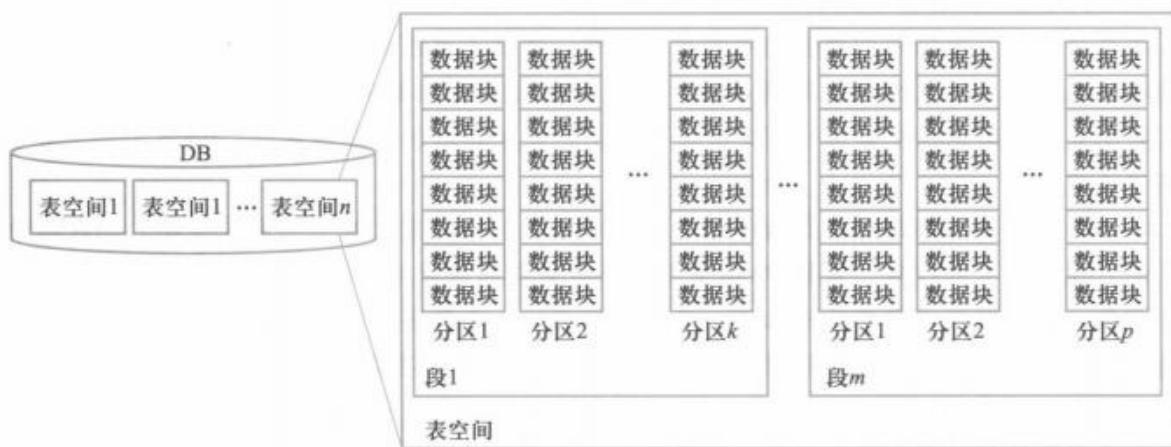


图 9.1 一种数据库的逻辑组织方式

在图 9.1 中，一个表空间对应磁盘上一个或多个物理文件，但一个物理文件只能属于一个表空间。一个数据库可以有多个表空间，从逻辑上组织数据库中的数据存储，如系统表空间、联机表空间、临时表空间等。一个表空间逻辑上由多个段组成，每个段可以逻辑上组织不同类型的数据，如数据段、索引段、临时段等。一个段逻辑上又由多个分区组成，每个分区由一组连续的数据块组成。块是数据库的磁盘存取单元，其大小为操作系统块的整倍数。

从逻辑上划分数据库空间是为了方便数据的管理。从物理上看，数据库中的数据最终是以文件（关系表）的形式存储在磁盘上。每个文件物理上分成定长的存储单元，即操作系统的物理块。物理块是存储分配和 I/O 处理的基本单位。一个物理块可以存放表中的多个记录（元

组), 一个表通常会占用多个块。因此数据库的物理组织形式是“文件-块-记录”。图 9.2 是逻辑组织方式与物理组织方式之间的对应关系。接下来, 9.1.2 小节将具体介绍如何组织单条记录, 9.1.3 小节将介绍如何在一个块中存放一组记录, 9.1.4 小节将介绍如何组织一个关系表在块中的存储。



图 9.2 数据库逻辑组织方式与物理组织方式的对应关系

9.1.2 记录表示

关系表的记录可以以定长记录和变长记录两种形式来存储。

1. 定长记录存储

以定长记录方式存储数据, 指的是关系表中每条记录占据相同大小的空间。即使表中有变长字段, 也可以以定长记录方式存储, 这时会为变长字段预留最大长度的空间。

例如, 对于第 3 章例 3.5 创建的 Student(Sno, Sname, Ssex, Sbirthdate, Smajor) 表, 采用定长记录存储方式, 其一条记录的存储形式如图 9.3 所示。这里给 VARCHAR 数据类型的 Sname、Smajor 属性分配了最大长度的空间。

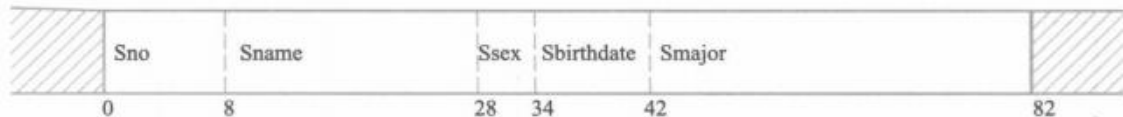


图 9.3 定长记录存储

有些硬件系统对内存中数据的起始地址有要求, 比如 4 的倍数或 8 的倍数。如果数据在磁盘上如图 9.3 这样紧密存储, 在读入内存时就需要做地址转换。为了简化地址转换, 并方便系统在不同平台上移植, 可以在外存中也保证各字段的起始地址是 4 或 8 的倍数, 如图 9.4 所示。在图 9.4 中, 每个字段都是从 8 的倍数的地址起始, 虽然 Sname 的定义是 20 B, 但实际分配给它 24 B。同样, 本来定义了 6 B 的 Ssex 属性, 这里分配了 8 B。

采用定长记录存储方式的好处是, 能快速定位到记录及其属性的物理位置, 增、删、改比较方便和快捷; 但这种方式显然会浪费一些存储空间。

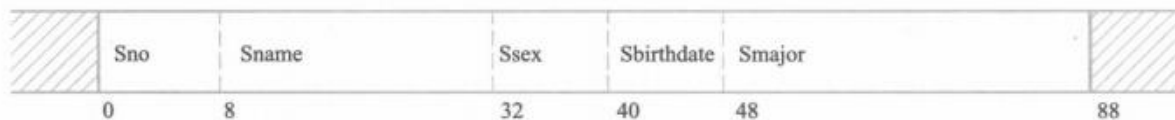


图 9.4 定长记录存储(字段起始地址为 8 的倍数)

2. 变长记录存储

Student 表的 Sname 属性和 Smajor 属性都是变长字符串, 所以 Student 表也可以以变长记录形式存储, 以节省存储空间。在组织变长记录时, 应保证能够快速访问一条记录, 并能快速地访问其中的所有属性, 包括定长和变长属性。通常有三种存储变长记录的方式。

第一种方式是在每条记录的头部记录该条记录的长度, 并在这条记录的每个变长字段前记录该字段的长度, 如图 9.5 所示。在这个例子中, 假设中文字符采用 GBK 编码, 每个汉字占 2 B。



图 9.5 变长记录存储(加长度标记)

第二种方式是先存储定长字段, 后存储变长字段, 第一个变长字段紧随定长字段之后, 从第二个变长字段开始, 在记录首部用指针(偏移量)指向变长字段, 如图 9.6 所示。



图 9.6 变长记录存储(先存储定长字段后存储变长字段)

第三种方式是将定长字段与变长字段分开存储在不同的块中, 图 9.7 示例了这种存储方式, 但实际上这种方式更多应用于 BLOB^① 等类型数据的存储。这种存储方式的好处是, 如果经常访问的是定长字段, 则可以减少数据存取时的 I/O 数量。

① BLOB 即 binary large object, 二进制大对象

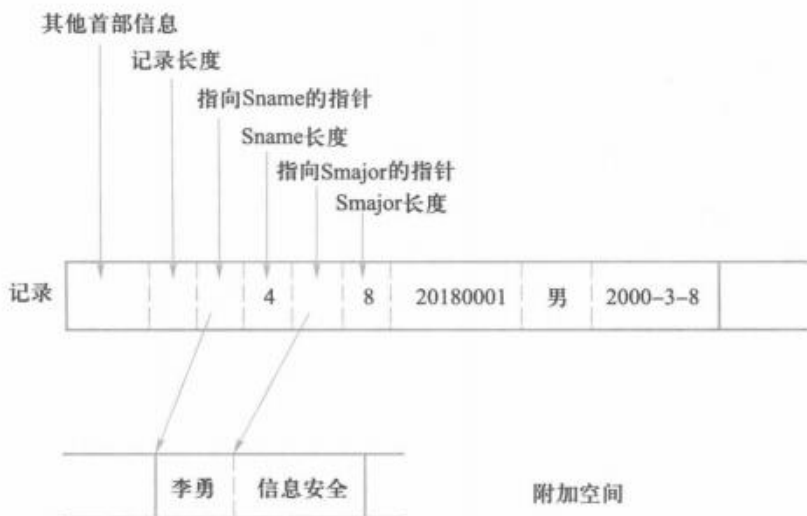


图 9.7 变长记录存储(定长字段与变长字段分开存储)

9.1.3 块的组织

定长记录和变长记录如何在块中组织存储呢?

1. 定长记录存储的块组织

定长记录的存储方式在实现上相对比较简单, 可以把一个表中的记录依次存放在块中。

图 9.8 是 Student 表采用定长记录存储方式时存放在一个物理块中的示意图, 每个块的首部可以有块头信息, 记录该块的 ID、最后一次修改和访问该块的时间戳、每条记录在块内的偏移量、空闲空间的头指针等。



图 9.8 定长记录存储的块组织

以定长方式存储记录, 需修改记录值时可直接在原位置修改, 不会出现原位置存放不下需要迁移记录的情形。删除记录时空间回收也很简单, 只需要将空闲空间加入空闲空间链表中即

可,不需要移动空闲空间,因为后面插入的新记录,其大小与被删除的记录是一样的,可以直接占用被删除记录的空间。图 9.9 是在图 9.8 所示的 Student 表中删除 20180003 和 20180006 两条记录后的情形。



图 9.9 定长记录存储的块组织(删除记录后)

2. 变长记录存储的块组织

对于变长记录的存储,块的组织就要稍复杂一点了,为了能够在块中快速定位到一条变长记录,通常需要在块头部分存放各条记录的指针(块内偏移量),空闲空间集中存储在块的中间部分,同时在块头有一个空闲空间尾指针(块内偏移量),以便在插入记录时快速找到空闲空间,如图 9.10 所示。



图 9.10 变长记录存储的块组织

在块中组织变长记录的存储时,记录从块的尾部开始连续存放。插入新记录时,从空闲空间尾部为其分配空间,在偏移量表中记录该记录的起始位置,并调整空闲空间尾指针。

删除一条记录时,会在偏移量表中为该记录指针置删除标记,释放其所占用的空间,并移动物理位置在其前面的记录,以保证空闲空间连续。相应地,被移动的记录在偏移量表中的指针以及空闲空间尾指针也需做相应修改。由于物理块通常都比较小,移动记录的开销并不高。

修改记录时,如果修改后记录在原位置存放不下,也会带来记录的迁移。

9.1.4 关系表的组织

了解了如何在一个块中存储一组记录之后,接下来的问题是如何组织关系表的存储。一个块只存储一个表的记录还是可以同时存储多个表的记录,这些记录按什么顺序存放,这些都是本节要讨论的问题。目前常见的关系表组织方式包括以下几种:堆存储、顺序存储、多表聚簇存储、B+ 树存储和哈希存储等。

1. 堆存储

表中的一条记录可以存储在该表的任何块中,没有顺序要求。在插入记录时,只要在该表的块中找到合适的空闲空间即可。如果没有足够的空闲空间,就为该表申请新的块。

2. 顺序存储

这种方式是一个表中的各条记录按照指定的属性或属性组的取值大小顺序存储。在一个块内,记录按照排序属性(组)的取值物理排列,同一个表的不同块之间则通过指针链接实现有序。这种组织方式可以高效地处理按排序属性(组)进行查询的请求。例如,如果 Student 表按专业(Smajor)升序存储,则可以快速完成如下的 SQL 语句:

```
SELECT Sno, Sname, Ssex  
FROM Student  
WHERE Smajor='计算机科学与技术';
```

但这种方式在插入或修改记录时,为保持记录顺序,有可能需要在块内或块间迁移已有记录,因而代价较高。

3. 多表聚簇存储

在默认情况下,一个块只用来存储同一个表中的记录,不同表的记录分别存储在不同的块中。但在有些情况下,不同表的记录也可以聚簇存储在同一组块中,其目的是减少连接操作的开销。

例如,两个具有主外码参照关系的表 Student 和 SC 可以按照学号(Sno)相等进行聚簇存储。图 9.11 是这种存储方式的一个示意图,在每个 Student 记录之后是该学生的选课记录。

如果用户想查询 20180001 学生的学号、姓名、选课门数,其 SQL 语句如下:

```
SELECT SC. Sno, Sname, COUNT(*)  
FROM Student, SC  
WHERE Student. Sno=SC. Sno AND SC. Sno='20180001';
```

那么只需要先找到 Student 表的 20180001 记录,其后跟着的就是 SC 表中该学生的所有选课记录,对其进行计数即可得到查询结果。明显地,对 Student 表和 SC 表按 Sno 值相等聚簇存储,相当于对 Student 表和 SC 表进行了预连接,它能够加速 Student 表和 SC 表的连接查询。

但是聚簇存储也是一把“双刃剑”,如果用户想查询数据科学与大数据技术专业的所有女生信息:



图 9.11 多表聚簇存储示意图

```
SELECT *
FROM Student
WHERE Smajor='数据科学与大数据技术' AND Ssex='女';
```

其效率就会低于 Student 表单独存放的情形，因为 Student 的元组会分散在更多的块里，需要更多次 I/O 操作。此外，这种数据组织方式在更新操作时会带来更频繁的数据迁移。例如，新学期开始时所有学生都新选了多门课程，为了保证每个学生的选课记录跟在其学生信息之后，将会频繁地迁移记录，以腾出空间插入新的选课记录。

4. B+树存储

传统的顺序存储方式在对表中记录进行增、删、改操作时，为维护记录顺序会付出较大的代价。B+树存储是以 B+树索引的形式确定记录存储在哪个数据块中，这样能在保持较高的记录访问效率的同时降低数据维护开销。B+树索引将在 9.2.3 小节介绍。与 B+树索引的区别是，B+树存储方式中叶节点的数据块中存放的不是索引项，而是数据记录。

5. 哈希存储

哈希存储方式是用哈希函数计算表中指定属性的哈希值，以此确定相应记录存储在哪个块中。哈希存储有两个关键要素，哈希函数和哈希表。哈希表由 B 个哈希桶(bucket)组成，每个桶对应一个或多个物理块，并用一个 0~B-1 之间的整数作为桶编号，桶中可以存储一条或多条记录。哈希函数以记录的哈希属性为输入，为其计算出一个 0~B-1 之间的整数，这个整数对应的是桶号。采用这种技术，可以将记录的存储位置与其指定属性的取值之间建立一个对应关

系,即按照记录的哈希属性值对应到一个存储位置,哈希属性值相同的记录会存储在相同的数据块中。在查找数据记录时,通过计算其哈希属性值可以快速定位到这条记录的位置。

9.2 索引结构

在很多情况下,用户的查询请求只涉及表中的少量记录,例如,查询计算机科学与技术专业的全体学生、查询李勇的出生日期等。当 Student 表很大时,如果读出该表的全部元组再逐条判断是否满足查询条件就会比较低效。

可以想象一下,当在一本书中查找某个特定的概念时,比较快捷的方法是通过该书的目录直接定位到想要查找的内容。如果关系表也能有一个类似于图书目录的附加结构,显然可以提高查询处理效率。这个附加结构就是索引。

索引的优势在于以下几个方面:

- ① 一个表的索引块数量通常比数据块数量少得多,因而搜索起来就会比较快。
- ② 索引通常采用一些易于检索的数据结构,可以使用高效的方法在索引中快速查找。
- ③ 对于经常访问的数据库表,如果其索引文件足够小则可以让其长久地驻留在内存缓冲区中,从而减少了 I/O 操作。

当然,索引也会带来额外的开销,包括存储空间的开销、建立索引的开销,以及当对基本表进行增、删、改操作时维护索引的开销。

数据库中常见的索引结构包括顺序表索引、辅助索引、B+树索引、哈希索引和位图索引等。下面依次介绍这几种索引结构。

9.2.1 顺序表索引

前面介绍了关系表的顺序存储方式。为了提高顺序表的查找效率,可以在顺序表的排序属性(组)上建立索引。这种索引结构是建在按索引属性值顺序存储的关系表上的,也称作主索引(primary index)或聚簇索引(clustering index)。它实际上就是第3章和第7章中提到的聚簇索引的一种形式。

注意:虽然称作主索引,但这种索引结构与主码(primary key)没有关系,索引属性可以是任意属性(组)。

顺序表上的主索引分为稠密索引和稀疏索引两类。索引结构中,每个索引项由索引属性的取值以及指向相应记录的指针两部分组成。与主索引相对应的是辅助索引(secondary index),将在9.2.2小节进行介绍。

1. 稠密索引

稠密索引(dense index)的索引块中存放每条记录的索引属性值以及指向相应记录的指针。以 Student 表 Sno 属性上的稠密索引为例,为方便举例,假设每个块只能存放2条 Student 表记录或6个 Sno 索引项,其索引如图9.12所示。其中,每条 Student 记录在索引中都对应了一个索引项。